StreetCare AI — data architecture: entity-relationship model

Left: the business entities in the delivered files and reference data · Right: the physical tables the service writes, with every column and type — crow's-foot notation

Built from data/*.csv · config/config.py · persistence/models.py · checkpoint.db schema

A Source & reference model *conceptual — CSV files, config and PDFs*

District *config*

PK	district_name	4 districts
	streets	12 each
	inspectors	2 appointed

Street *48 streets*

PK	street_name	Station Road
FK	district	→ District
	street_category	Main route …

StreetWorksPermit *register*

PK	permit_reference	SWP0071263
FK	street_name	→ Street
	works_location	
	works_promoter	
	works_description	
	permit_status	Live / Cancelled

R1 District 1 — N Street · R2 Street 1 — 0..N StreetWorksPermit · R3 Street 1 — 0..N DefectReport

DefectType *5 types*

PK	defect_type	Footway …
FK	standard_document	→ PDF
	requirement kinds	per standard

DefectReport *history + open queue*

PK	report_reference	SC0052158
PK	revision_number	latest kept
	report_channel · reporter_type	
FK	street_name	→ Street
	district · street_category	copied
	location_description	
	location_fixed · photograph_attached	Yes / No
FK	defect_type	→ DefectType
FK	linked_defect_type	0..1 second
	defect_depth_mm · defect_width_mm	mm
	water_depth_mm · water_reach_m	mm · m
	water_location · overhang_location	
	clear_height_m · on_highway_land	
	lighting_fault · column_number_given	
	column_lean_degrees · column_height_m	
	previous_reports_at_location	
	planned_crew_hours	h
	live_ / conflicting_street_works	history
	review_outcome · repeat_cause	history
	reported_again	target
	supporting_document_count	open only

RepairLog *1 per report*

PF	report_reference	→ report
	repair_method	
	traffic_management	
	crew_size	
	planned_crew_hours	h
	completion_code	
	actual_crew_hours	target
	reported_again	

R4 DefectType 1 — 0..N DefectReport · R5 DefectType 1 — 1..N Requirement · R6 DefectType 1 — 1 PolicyDocument · R7 DefectReport 1 — 0..1 RepairLog · R8 DefectReport 1 — 0..N ReportDocument

Requirement *10 kinds*

PK	requirement_kind	depth …
FK	defect_type	→ DefectType
	requirement_name	
	requirement_sentence	quoted
	rule figures	exact decimals

ReportDocument *attached*

PK	document_id	DOC-841228
FK	report_reference	→ report
	document_type	3 kinds
	document_title	
	document_text	

PolicyDocument *16 PDFs*

PK	document_name	*.pdf
	kind	standard · guide …
	numbered sections	1 … n

same document types

StatementSentence *corpus*

PK	sentence_id	
	report_reference	own refs
	document_type	
	sentence_number	
	sentence_text	
	sentence_length_characters	
	sentence_type	target · 9

R9 PolicyDocument 1 — 1..N KnowledgePassage

KnowledgePassage *205 in ChromaDB*

PK	id	
FK	document_name	→ PolicyDocument
	section_number	
	passage_number	order
	carried_characters	overlap
	document (text)	≤ 700 chars
	embedding	384-d MiniLM

Relationships

R1	**District 1 — N Street**	12 streets per district (INSPECTION_DISTRICT_STREETS)
R2	**Street 1 — 0..N StreetWorksPermit**	street_name; Live permits count, Cancelled do not
R3	**Street 1 — 0..N DefectReport**	street_name; the street fixes the deciding district
R4	**DefectType 1 — 0..N DefectReport**	defect_type, and again (dashed) via linked_defect_type
R5	**DefectType 1 — 1..N Requirement**	parsed from that type's standard PDF at run time
R6	**DefectType 1 — 1 PolicyDocument**	its Defect-Standard-<type>.pdf
R7	**DefectReport 1 — 0..1 RepairLog**	report_reference; the log holds actual crew hours
R8	**DefectReport 1 — 0..N ReportDocument**	report_reference; 60 documents for 40 open reports
R9	**PolicyDocument 1 — 1..N KnowledgePassage**	document_name + section_number for citations

Checked on the data

- report_reference is unique in every clean file once the latest revision is kept (highest revision 3)
- 99.3 % of repair-log references also appear in report history — R7 is optional, not mandatory
- statement sentences are a separate labelled corpus: 6.1 per reference on average (max 15), none in history
- live queue: 40 open reports, 60 documents, 28 permits (23 live) on 48 streets in 4 districts

Model inputs: DefectReport + counts screened under each Requirement + trigger flags; targets reported_again, actual_crew_hours, sentence_type.

B Operational model *physical — database/streetcare.db (SQLAlchemy) and database/checkpoint.db (LangGraph)*

user_accounts *7 cols · 16 seeded*

PK	username	VARCHAR
	display_name	VARCHAR
	role	VARCHAR
	district	VARCHAR NULL
	password_hash	VARCHAR
	password_salt	VARCHAR
UQ	session_token	VARCHAR NULL

opens

agent_runs *7 cols*

PK	run_id	VARCHAR
UQ	thread_id	VARCHAR
FK	coordinator_id	VARCHAR
	report_reference	VARCHAR
	coordinator_request	TEXT
	status	VARCHAR
	llm_call_count	INTEGER

logs 5–7 calls

llm_calls *8 cols*

PK	call_id	INTEGER
FK	run_id	VARCHAR
	agent_name	VARCHAR
	model_name	VARCHAR
	prompt_tokens	INTEGER
	response_tokens	INTEGER
	total_tokens	INTEGER
	latency_milliseconds	INTEGER

asks

chat_turns *12 cols*

PK	turn_id	INTEGER
FK	username	VARCHAR
	user_role	VARCHAR
	question_text	TEXT
	question_type	VARCHAR
	report_reference	VARCHAR NULL
	street_name	VARCHAR NULL
	defect_type	VARCHAR NULL
	answer_text	TEXT
	source_documents	TEXT
	closest_distance	FLOAT
	was_refused	BOOLEAN

decides (decided_by)

report_reviews *37 cols*

PK	review_id	VARCHAR
UQ	run_id	VARCHAR
	report_reference	VARCHAR
	revision_number	INTEGER
	defect_type	VARCHAR
	linked_defect_type	VARCHAR
	street_name	VARCHAR
	district	VARCHAR NULL
	coordinator_request	TEXT
	requirements	JSON
	requirements_met_count	INTEGER
	requirements_not_met_count	INTEGER
	requirements_not_documented_count	INT
	live_permits	JSON
	conflicting_permits	JSON
	coordination_triggers	JSON
	assessment	VARCHAR
	decisions_open	JSON
	evidence	JSON
	disagreements	JSON
	repeat_likelihood	FLOAT
	likelihood_band	VARCHAR
	expected_crew_hours	FLOAT
	planned_crew_hours	FLOAT NULL
	overrun_expected	BOOLEAN NULL
	crew_hours_text	TEXT
	review_plan_text	TEXT
	requirements_analysis_text	TEXT
	evidence_review_text	TEXT
	prediction_analysis_text	TEXT
	recommendation_text	TEXT
	recommended_decision	VARCHAR
	status	VARCHAR
	order_document_text	TEXT NULL
	document_details	JSON NULL
	resident_update_text	TEXT NULL
	coordination_brief	JSON NULL

thread_id: one paused / finished run

review_decisions *8 cols*

PK	decision_id	INTEGER
UQ	review_id	VARCHAR
FK	run_id	VARCHAR
	decision	VARCHAR
FK	decided_by	VARCHAR
	decided_by_role	VARCHAR
	decided_by_district	VARCHAR
	decision_reason	TEXT

database/checkpoint.db

checkpoints *7 cols*

PK	thread_id	TEXT
PK	checkpoint_ns	TEXT
PK	checkpoint_id	TEXT
	parent_checkpoint_id	TEXT
	type	TEXT
	checkpoint	BLOB
	metadata	BLOB

writes *9 cols*

PK	thread_id	TEXT
PK	checkpoint_ns	TEXT
PK	checkpoint_id	TEXT
PK	task_id	TEXT
PK	idx	INTEGER
	task_path	TEXT
	channel	TEXT
	type	TEXT
	value	BLOB

Status columns — allowed transitions

agent_runs.status

RUNNING → AWAITING_DECISION → COMPLETE

FAILED (502)

report_reviews.status

AWAITING_DECISION → REPAIR_ORDERED | INSPECTION_REQUESTED

REFERRED_FOR_COORDINATION

one decision only

Relationships and bridges to model A

- **user_accounts 1 — 0..N agent_runs** — coordinator_id = username (the coordinator who opened it)
- **agent_runs 1 — 1 report_reviews** — run_id, unique in both; joined inner in fetch_review_records
- **report_reviews 1 — 0..1 review_decisions** — review_id, unique; joined outer, so undecided reviews show
- **agent_runs 1 — 5..7 llm_calls** — run_id; 5 before the gate, then 7 ORDER · 6 INSPECT · 5 REFER
- **user_accounts 1 — 0..N review_decisions** — decided_by = username; never the opener (four-eyes)
- **user_accounts 1 — 0..N chat_turns** — username; every reply stored, refusals too
- **agent_runs 1 — 1 checkpoint thread** — thread_id = checkpoints.thread_id = writes.thread_id
- **report_reviews → DefectReport (A)** — (report_reference, revision_number): one review per revision
- **review_decisions → District (A)** — decided_by_district must hold the report's street
- **chat_turns → DefectReport · Street · DefectType · PolicyDocument (A)** — report_reference, street_name, defect_type; source_documents names the PDFs

No FOREIGN KEY constraints are declared: persistence/repository.py, the only module that queries, joins on these keys. Grey dots link to model A.

exactly one · zero or one · one or many · zero or many · PK primary key · PF primary + foreign key · UQ unique · FK logical reference · related, not a key

access · review & decision · LLM-written / logged · reference / run · labelled training data